

Cloud Infrastructures

07 - Container orchestration with Kubernetes

Juan Ángel Lorenzo del Castillo

CY Cergy Paris Université

ING3 - Ingénierie du Cloud Computing

2024-2025



juan-angel.lorenzo-del-castillo@cyu.fr

Table of Contents

- 1 Container orchestration
- 2 Kubernetes
- 3 Kubernetes architecture and terminology
- 4 Tutorial

Table of Contents

- 1 Container orchestration
- 2 Kubernetes
- 3 Kubernetes architecture and terminology
- 4 Tutorial

Container orchestration

- You often start with running one container on one host. You quickly discover how easy it is to run new versions of software, how easy it is to share that software with others, and how easy it is for those users to run it the way you intended.
- But then you need
 - ▷ Two containers
 - ▷ Two hosts
- It's easy to fire up one web server on port 80 with a container, but what happens...
 - ▷ when you need to fire up a second container on port 80?
 - ▷ when you are building a production environment and you need the containerized web server to fail over to a second host?

Container orchestration

- When running 60 or 70 services, how would you manage...
 - ▷ load balancers
 - ▷ clustering software
 - ▷ clustered file systems
 - ▷ adding a new service
 - ▷ decommissioning a service
 - ▷ etc.

You need a kind of supervisor program, which continually checks that your containers are running, and if they ever stop, starts them again immediately.

You need to move into ***container orchestration***.

Container orchestration

- Today's most promising emerging technologies are pairing containers with cluster management software such as:
 - ▷ **Docker Swarm** (<https://docs.docker.com/engine/swarm/>)
 - ▷ **Apache Mesos** (<http://mesos.apache.org>)
 - ▷ **Kubernetes** (<https://kubernetes.io/>)

Table of Contents

- 1 Container orchestration
- 2 **Kubernetes**
- 3 Kubernetes architecture and terminology
- 4 Tutorial

Kubernetes?



kubernetes

Kubernetes is an open-source orchestration system for managing clusters of containers.

- ▶ It provides tools for deploying and scaling applications as needed, i.e. specify how many machines or containers we want to keep running, how to scale them...
- ▶ Can manage changes to existing containerized applications
- ▶ We can specify how to connect containers or machines to other containers and persistent storage.
- ▶ It can deploy a group of containers as a unit.
- ▶ It helps optimising the use of the underlying hardware beneath your containers.
- ▶ Designed to be extensible and fault-tolerant.

- Kubernetes **is not** a PaaS, but a more basic framework allowing users to choose languages, monitoring and logging tools, etc. It can be used as the basis for a complete PaaS to run on top of it.

Kubernetes



"So let me get this straight. You want to build an external version of the Borg task scheduler. One of our most important competitive advantages. The one we don't even talk about externally. And, on top of that, you want to open source it?"

Urs Holzle, head of technical infrastructure and chief architect of Google Cloud.

A bit of history:

- **Borg**: Google's original cluster management system. Runs hundreds of thousands of jobs on containers at their data centres at high utilisation.
- Borg was used to deliver the Google Cloud Platform and its IaaS, **Google Compute Engine**. Clients were underutilizing the infrastructure, because they were running their services on VMs.
- Turned to Docker, incorporated their knowledge about Borg, **open-sourced it** and combined it with an elegant and easy-to-use UI: **kubernetes**

What is Kubernetes?

Kubernetes is a dump truck



Image credits : Wikimedia, CC BY-SA 2.0

Advantages:

- They can move dirt, gravel, rocks, coal, construction material, or road barricades.
- They can even pull trailers with other pieces of heavy equipment on them.
- You can load a dump truck with five tons of dirt and drive across the country with it.
- And they do it in an *elegant* way.

Drawbacks:

- Not easy to use.
- Require a special driver's license.
- Not easy to equip.
- There are a ton of options when you buy a dump truck.
- There is a lot of maintenance.

What is Kubernetes?

Kubernetes is a dump truck



Image credits : Wikimedia, CC BY-SA 2.0

- If you owned a construction company, you would not build your own dump trucks.
- You would not maintain the supply chain to rebuild dump trucks.

What is Kubernetes?

Kubernetes is a dump truck



Image credits : Wikimedia, CC BY-SA 2.0

- If you owned a construction company, you would not build your own dump trucks.
- You would not maintain the supply chain to rebuild dump trucks.
- But you would learn to drive one.

What is Kubernetes?

Kubernetes is a dump truck



Image credits : Wikimedia, CC BY-SA 2.0

- If you owned a construction company, you would not build your own dump trucks.
- You would not maintain the supply chain to rebuild dump trucks.
- But you would learn to drive one.

We will not learn how to build and equip kubernetes, but how to **utilise** kubernetes.

What is Kubernetes?

Managed Kubernetes Services:

- Google Kubernetes Engine (GKE)
- Amazon Elastic Container Service for Kubernetes (EKS)
- Azure Kubernetes Service (AKS)
- OpenShift
- IBM Cloud Kubernetes Service
- Heptio Kubernetes Subscription (HKS)

Turnkey Kubernetes Solutions (ready-to-use K8s cluster):

- Stackpoint
- Containership Kubernetes Engine

Kubernetes installer (self-hosted):

- | | |
|-------------|-----------------------------|
| ■ kops | ■ Tarmak |
| ■ Kubespray | ■ Rancher Kubernetes Engine |
| ■ TK8 | ■ Puppet Kubernetes Module |
| ■ Kubeadm | ■ Kubeformation |

Table of Contents

- 1 [Container orchestration](#)
- 2 [Kubernetes](#)
- 3 [Kubernetes architecture and terminology](#)
- 4 [Tutorial](#)

Kubernetes states

- Kubernetes works on the principles of *defined state* and *actual state*.

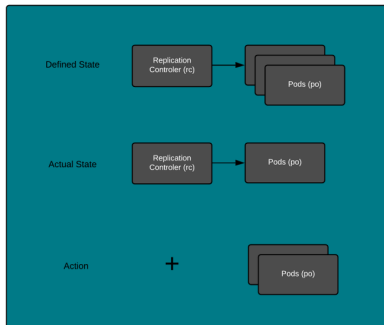


Image credits : Scott McCarty - opensource.com

1. Developers, sysadmins or operators specify the defined state using the YAML/JSON files they submit to the Kubernetes API.
2. Then, Kubernetes uses a controller to analyse the difference between the new state defined in the YAML/JSON and the actual state in the cluster.

Architectural components

- A **Pod** is a type of job running in a Kubernetes cluster.
 - ▷ It runs one or more containers.
 - ▷ They share networking and storage separate from the node (they have their own IP).
 - ▷ The containers in a pod are considered “tightly coupled”, and they are managed and deployed **as a single unit**. It is analogous to a VM.
 - ▷ Data is stored in volumes in memory or persistent disks.

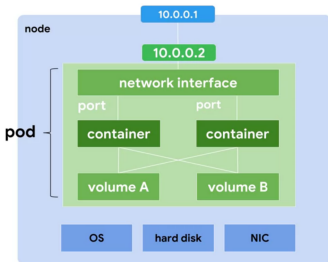


Image credits : Google



Image credits : kubernetes.io

Architectural components

- **Nodes** (also called *minions*) represent a **worker machine** in a Kubernetes cluster. The worker machine can be either physical or (more likely) virtual. A node contains all the required services **to host a pod**.
- Components:
 - ▶ **Kubelet:** agent for managing the node, starting and monitoring workloads, and communicating with the Kubernetes master.
 - ▶ **kube-proxy:** Routes requests between Pods on different nodes, and between Pods and the internet.
 - ▶ **Container runtime:** starts and stops containers and handles their communications. Docker or other.

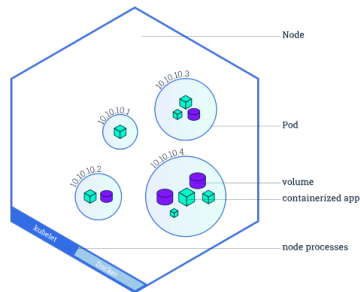


Image credits : kubernetes.io

Architectural components

- A cluster always requires a **master** node, where the **controlling services** (known as the **master components**) are installed. These services can be distributed on a single machine or across multiple machines for redundancy. They control communications, workload, and scheduling. The master schedules the containers to run on the cluster's nodes. The nodes communicate with the master using the **Kubernetes API**, which the master exposes.

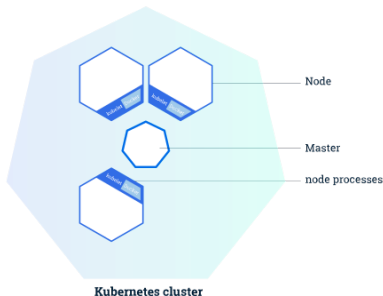


Image credits : kubernetes.io

Architectural components

- A **Service** is an abstract way to expose an application running on a set of Pods as a network service.
- Kubernetes assigns a fixed IP and a single DNS name to the pod replicas and allows other pods or services to communicate with them.
- A service is defined with a YAML file.
- There will be multiple services running with different configurations and features.
 - ▷ Usually, the frontend is a web server, application server, etc.
 - ▷ Backends are usually the nodes and maybe a database server.
 - ▷ They will usually apply a load balancer.

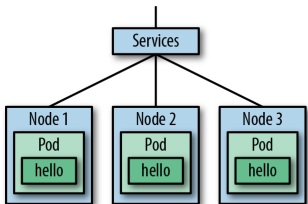


Image credits : Cloud Native DevOps with Kubernetes

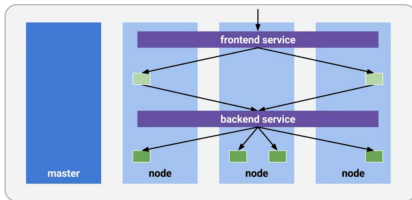


Image credits : Google

Labels and Selectors

- **Labels** are metadata (key-value pairs) without any pre-defined meaning that can be assigned to any API object and represent an identity.
- They are:
 - ▷ The only grouping mechanism for pods
 - ▷ Searched by selectors.
- Example:

```
"metadata": {  
  "labels": {  
    "key1" : "value1",  
    "key2" : "value2"  
  }  
}
```

Labels and Selectors

■ Example: four pods and three labels

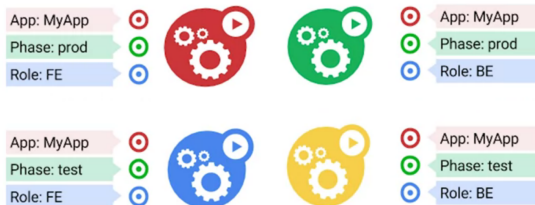
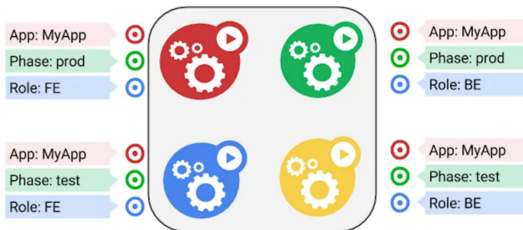


Image credits : Google

Labels and Selectors

- We can query for labels that map to a value like the entire app



App = MyApp

Image credits : Google

Labels and Selectors

- Or narrow your search with multiple labels (e.g. your app's frontend)



App = MyApp, Role = FE

Image credits : Google

Labels and Selectors

- or narrow for your app's backend



App = MyApp, Role = BE

Image credits : Google

Labels and Selectors

- or narrow for your app's test phase

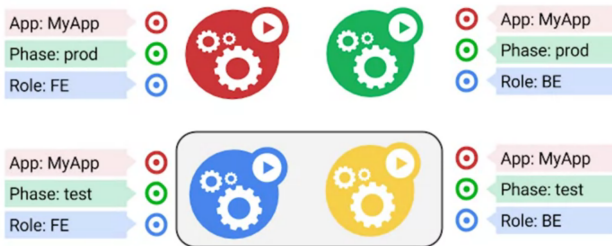


Image credits : Google

Labels and Selectors

- or narrow for your app's production release

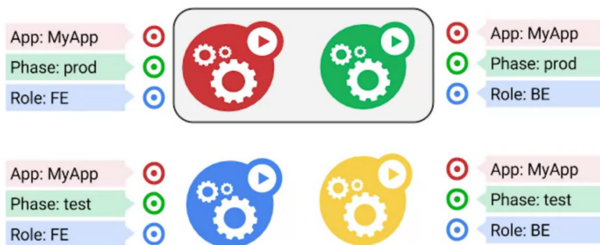


Image credits : Google

Namespaces

- **Namespaces** are a way of partitioning a cluster into separate subdivisions that are not visible to each other.
- Example:
 - ▷ A `test` namespace for testing, and a `prod` namespace for production applications.
 - ▷ We could have a service called `demo` in the `prod` namespace, and another `demo` in the `test` namespace.
- Check the namespaces on your cluster:

```
kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	13d
kube-node-lease	Active	13d
kube-public	Active	13d
kube-system	Active	13d

- If we do not specify anything, the `default` namespace will be used. We can use a namespace with the `-namespace` flag. For example:

```
kubectl get pods -namespace prod
```

Namespaces

- We can create a namespace in a declarative way, by using a `namespace.yaml` manifesto:

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

- and apply it with

```
kubectl apply -f namespace.yaml
```

Cluster overview

Cluster Overview

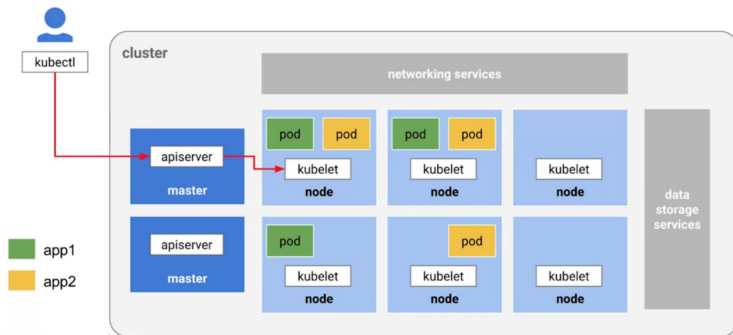


Image credits : Google

- ▶ Kubelets check the health of the pods on each node. If a pod is not healthy, they restart it.

Table of Contents

- 1 [Container orchestration](#)
- 2 [Kubernetes](#)
- 3 [Kubernetes architecture and terminology](#)
- 4 [Tutorial](#)

Tutorial

Hands on!

- Before starting with Kubernetes, let us create a toy container and upload its image to the DockerHub:

```
$ git clone https://github.com/cloudnativdevops/demo.git  
$ cd ./demo/hello  
$ docker image build -t myhello .  
$ docker container run -p 9999:8888 myhello
```

- ▷ Go to <http://localhost:9999> and check that it works.
- ▷ Delete the container and upload the image to DockerHub:

```
$ docker login  
$ docker image tag myhello YOUR_DOCKER_ID/myhello  
$ docker image push YOUR_DOCKER_ID/myhello
```


Tutorial

- Now let us install `Minikube`. `Minikube` sets up a local Kubernetes cluster in your computer for testing and learning purposes.

- ▶ Go to <https://minikube.sigs.k8s.io/docs/start> and follow Point 1 to install `Minikube` in your computer.
- ▶ Then, start it

```
$ minikube start
```

- ▶ We need `kubectl` in our cluster. Install it from the [Kubernetes site](#).

Tutorial

- Run the app inside a Pod in an **imperative** way:

```
$ kubectl run demo --image=YOUR_DOCKER_ID/myhello --port=9999 --labels app=demo
$ kubectl port-forward pod/demo 9999:8888
```

- You can list the available nodes and the running pods with:

```
$ kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
minikube      Ready     control-plane   12d   v1.31.0

$ kubectl get pods --selector app=demo
```

- Check that the app works by going to <http://localhost:9999>
- You can get information about the newly created Pod:

```
$ kubectl describe pod/demo
```

- Finally, you can delete the Pod with the command:

```
$ kubectl delete pod demo
```

Tutorial

Let us create a *Deployment*:

- From the demo project that you cloned from GitHub, have a look at `hello-k8s/k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo
  labels:
    app: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
        - name: demo
          image: cloudnativelabs/demo:hello
          ports:
            - containerPort: 8888
```

Tutorial

- Modify the number of *replicas* to 2.
- In a **declarative** way, submit this YAML manifest to the cluster:

```
$ cd hello-k8s
$ kubectl apply -f k8s/deployment.yaml
```

- After a few seconds, check that the deployment is ready and that two Pods are running:

```
$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
demo      2/2     2            2           17m

$ kubectl get pods --selector app=demo
NAME                                READY   STATUS    RESTARTS   AGE
demo-85dfc776bd-cr7v               1/1     Running   0          18m
demo-85dfc776bd-fd2rp              1/1     Running   0          18m
```

Tutorial

- We have set up the ReplicaSet to 2. If a Pod is down, the node's kubelet will spot this and will start a replacement to keep the **Desired State**:

```
$ kubectl delete pod demo-85dfc776bd-crn7v
pod "demo-85dfc776bd-crn7v" deleted

$ kubectl get pods --selector app=demo
```

NAME	READY	STATUS	RESTARTS	AGE
demo-85dfc776bd-5m2m	1/1	Running	0	6s
demo-85dfc776bd-fd2rp	1/1	Running	0	18m

- We can scale up or down the Deployment to a given number of replicas by hand. For more information, see Kubernetes documentation: [Running Multiple Instances of Your App](#).

Tutorial

Let us create a *Service*:

- In order to connect to the demo Pod with a web browser, we are going to create a Service. From the demo project that you cloned from GitHub, have a look at `hello-k8s/k8s/service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: demo
  labels:
    app: demo
spec:
  ports:
    - port: 8888
      protocol: TCP
      targetPort: 8888
  selector:
    app: demo
  type: ClusterIP
```

Tutorial

- Apply the manifest to create the Service:

```
$ kubectl apply -f k8s/service.yaml  
service/demo created
```

- Connect the demo service to a port on our machine.

```
$ kubectl port-forward service/demo 9999:8888  
Forwarding from 127.0.0.1:9999 -> 8888  
Forwarding from [::1]:9999 -> 8888
```

- If you go to <http://localhost:9999> you will see that you have access to the service.
- Finally, clean up everything:

```
$ kubectl delete -f k8s/  
deployment.apps "demo" deleted  
service "demo" deleted
```

Tutorial

To learn more:

If you want to know more about Kubernetes, the official documentation is a good starting point: [Kubernetes tutorial](#)